

SYSTÈME DE MESSAGERIE DISTRIBUÉE INSPIRÉ DE
L'INTRICATION QUANTIQUE

QUANTUM TWIN



DOUAA BEN MARZOUK - MARAM HADJ ALI - RACHED ILAHI - NOUR BEN TAHER

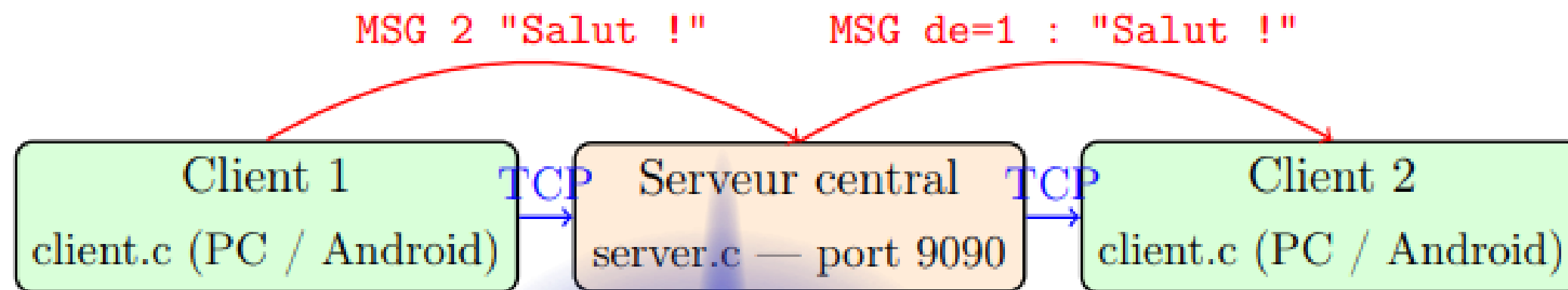


FIGURE 1 – Architecture initiale TCP — Phase 1

SERVEUR TCP

Serveur C (server.c) : socket TCP SOCK_STREAM, bind() sur port 9090, listen() + accept() en boucle infinie. Activation SO_REUSEADDR pour redémarrage sans erreurs de port occupé.

CLIENT & UDP

Client C (client.c) : socket TCP, connect() vers IP serveur, communication full-duplex. Mode UDP (SOCK_DGRAM) disponible en bonus pour commandes PING et STATE sans connexion persistante.

BRIDGE WEBSOCKET

Phase 2 : bridge.py assure le relais WebSocket ↔ TCP brut, permettant l'accès mobile universel. Le serveur C reste inchangé, seul le bridge gère la conversion de protocoles.



SOCKETS TCP/UDP LE CANAL DE COMMUNICATION

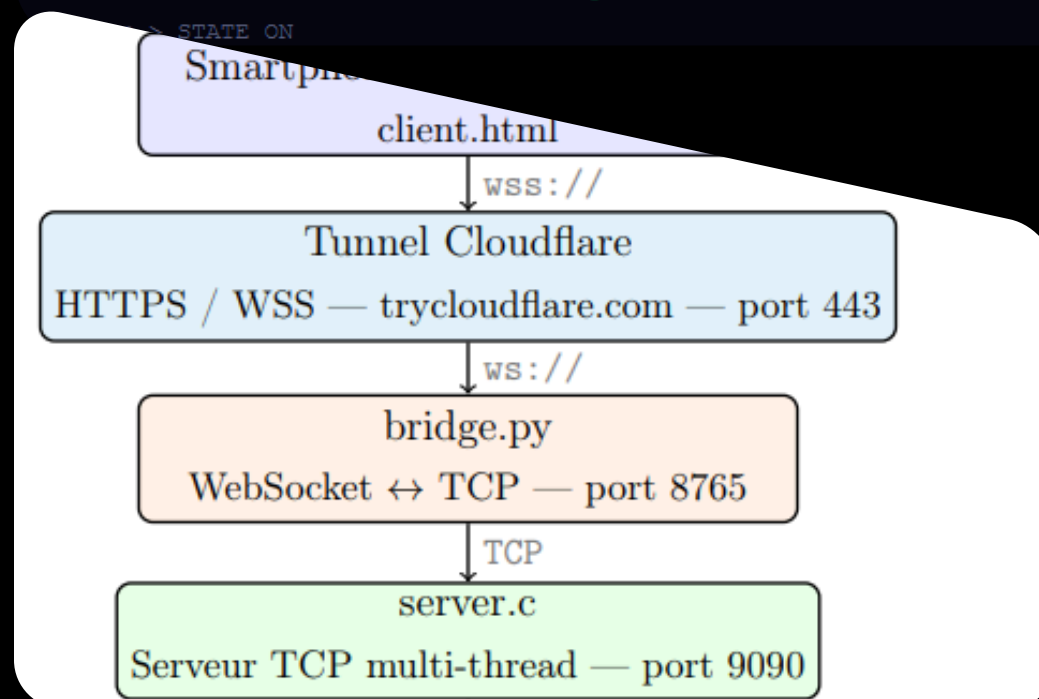


MULTITHREADING — UN THREAD PAR CLIENT

```
● QUANTUM TWIN
192.168.100.20:8765

TERM

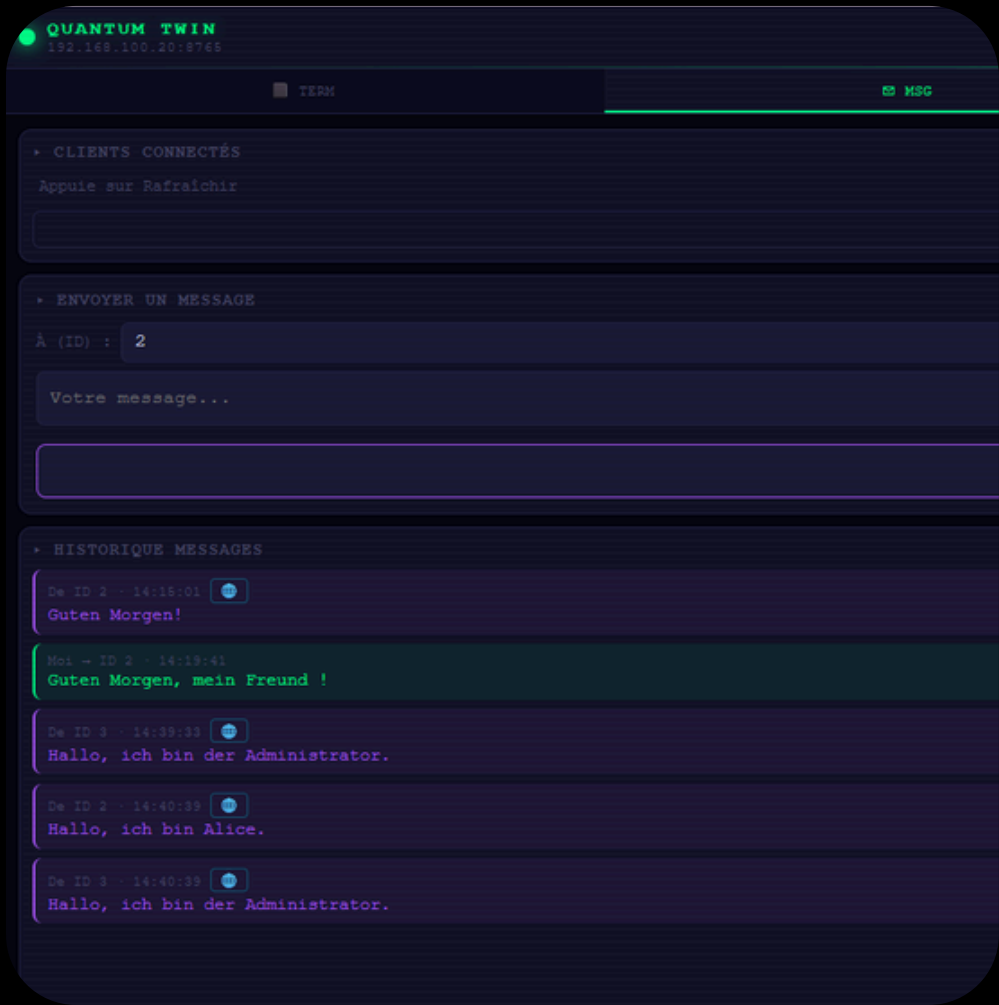
14:12:21 Comptes: alice/pass123 bob/pass456 admin/secret123
14:14:09 Connexion à 192.168.100.20:8765...
14:14:09 [Connecté via WebSocket]
14:14:09 > CONNECT bob pass456
14:14:09 ← OK CONNECTED id=1 role=user
14:14:09 [Mon ID: 1]
14:15:01 > LIST
14:15:01 ← ERROR Accès refusé (admin requis)
14:15:02 > LIST
14:15:02 ← ERROR Accès refusé (admin requis)
14:15:41 > MSG 2 Guten Morgen, mein Freund !
14:15:41 > LIST
14:15:41 ← OK MSG envoyé
14:15:41 ← ERROR Accès refusé (admin requis)
14:21:40 > LIST
14:21:40 ← ERROR Accès refusé (admin requis)
> STATE ON
```



ARCHITECTURE MULTITHREAD

pthread_create() : création d'un thread dédié par client accepté (max 64 connexions simultanées).
pthread_detach() : threads détachés automatiquement, pas besoin de join(). Thread réception client : écoute serveur en arrière-plan pendant saisie utilisateur.

Fonction client_thread() : réception → parsing commande → dispatch handler. Passage d'arguments via struct ThreadArg* (sockfd, IP, is_admin, client_id).
Avantage principal : serveur non bloquant sur clients lents ou inactifs.



MUTEX & PROTECTION
DES RESSOURCES

MUTEX	RESSOURCE	FONCTION
mtx_clients	✓	✗
mtx_pairs	✗	✓
mtx_histo	✓	✓
mtx_mem	✓	✓
save_clients()	✗	✓

SYNCHRONISATION



JUMEAUX NUMERIQUES



APPARIEMENT SYNC

Commande SYNC [twin_id] pour lier deux clients. Paires stockées dans pairs.txt avec persistance.



PROPAGATION STATE

Client A envoie STATE ON → find_twin() → UPDATE ON transmis au jumeau en temps réel.

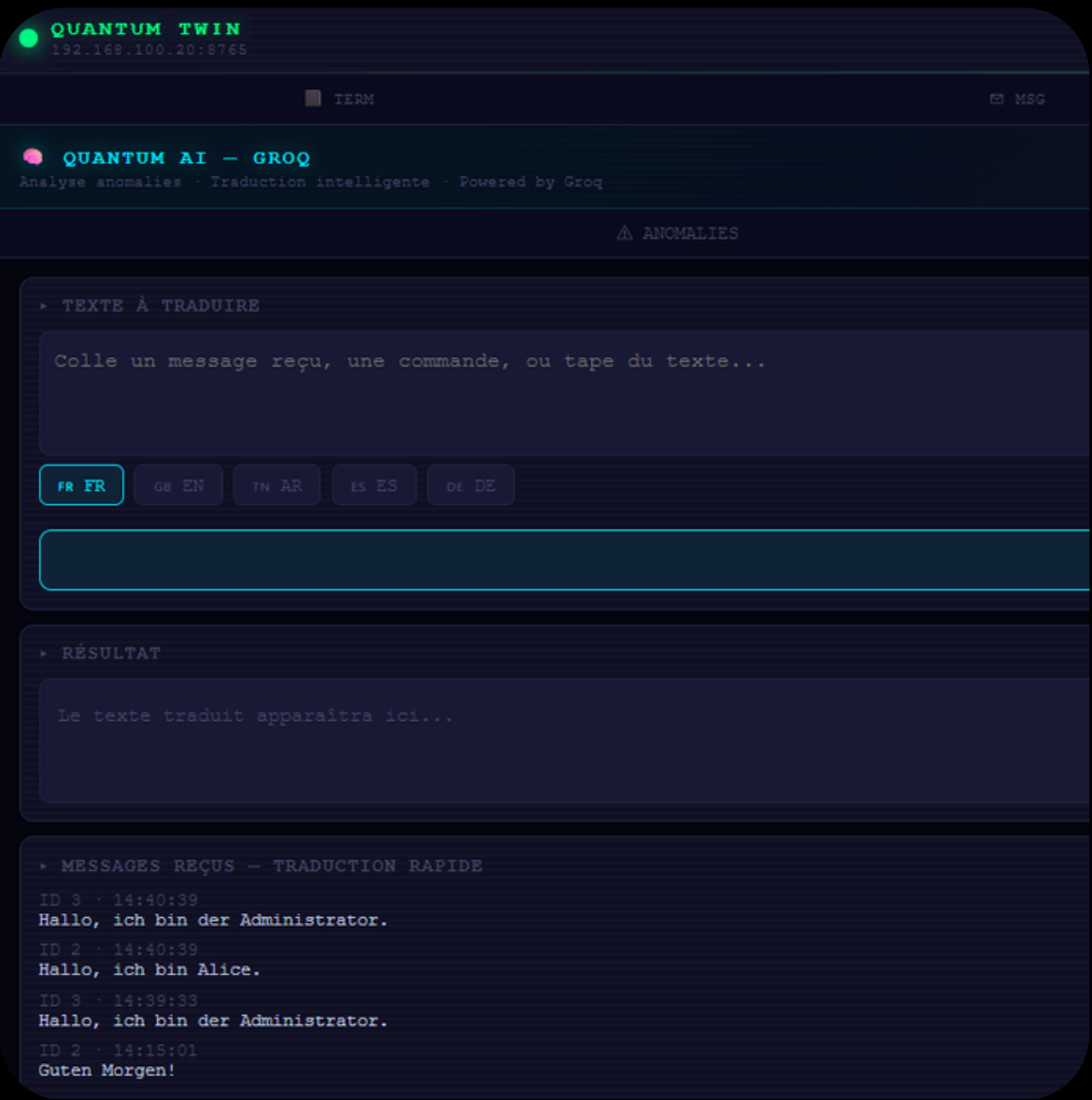
MESSAGERIE MSG

MSG [id] [texte] → handle_msg() → envoi via socket active du destinataire.

HISTORIQUE HISTO

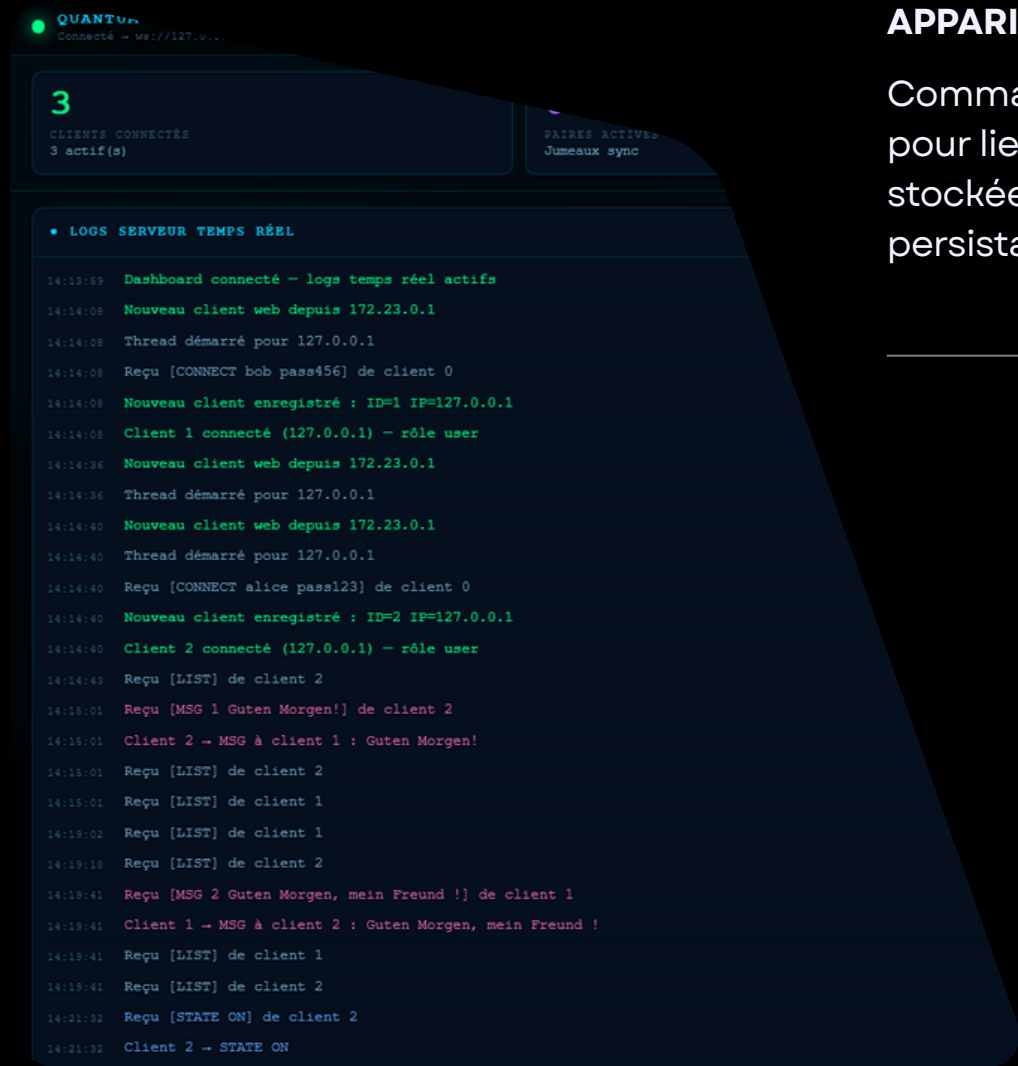
HISTO [n] / HISTO MY [n] pour consulter logs. Persistance histo.txt avec timestamp.

SYNCHRONISATION TEMPS RÉEL.





TYPES CLIENTS



APPARIEMENT SYNC

Commande SYNC [twin_id] pour lier deux clients. Paires stockées dans pairs.txt avec persistance.



PROPAGATION STATE

Client A envoie STATE ON → find_twin() → UPDATE ON transmis au jumeau en temps réel.

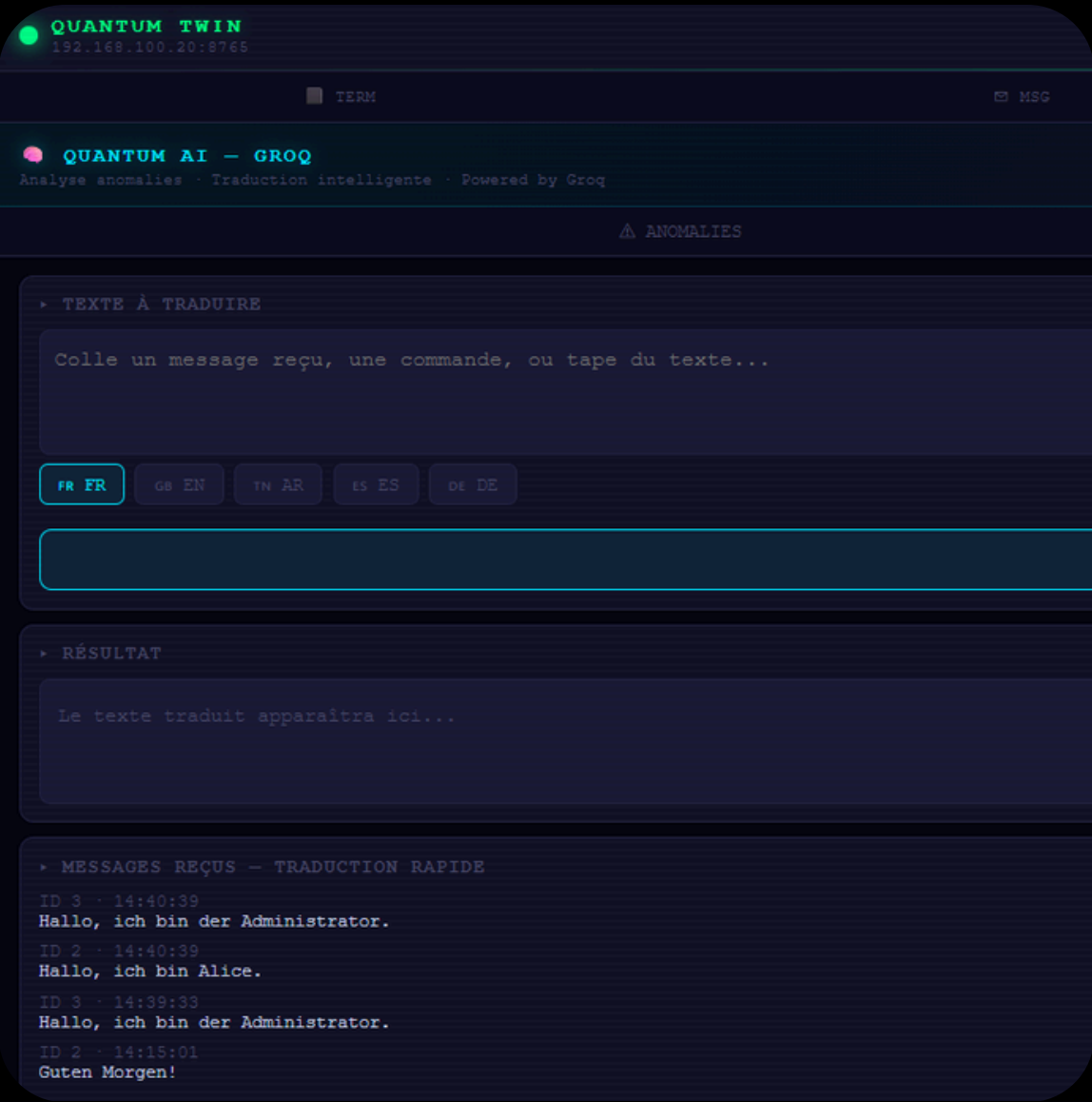
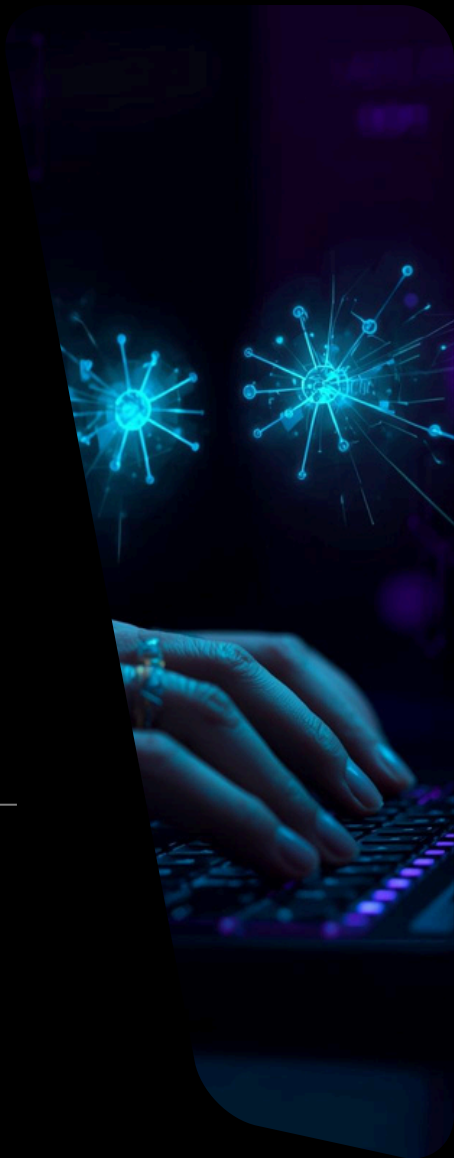
MESSAGERIE MSG

MSG [id] [texte] → handle_msg() → envoi via socket active du destinataire.

HISTORIQUE HISTO

HISTO [n] / HISTO MY [n] pour consulter logs. Persistance histo.txt avec timestamp.

SYNCHRONISATION TEMPS RÉEL.





IA

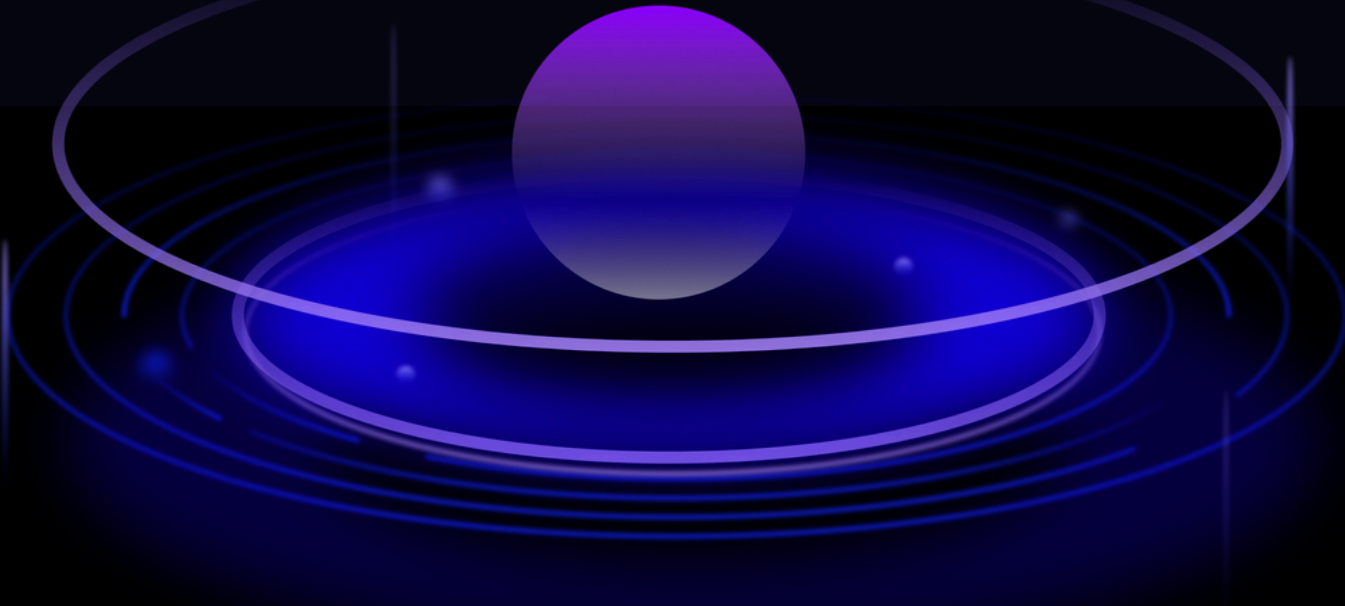
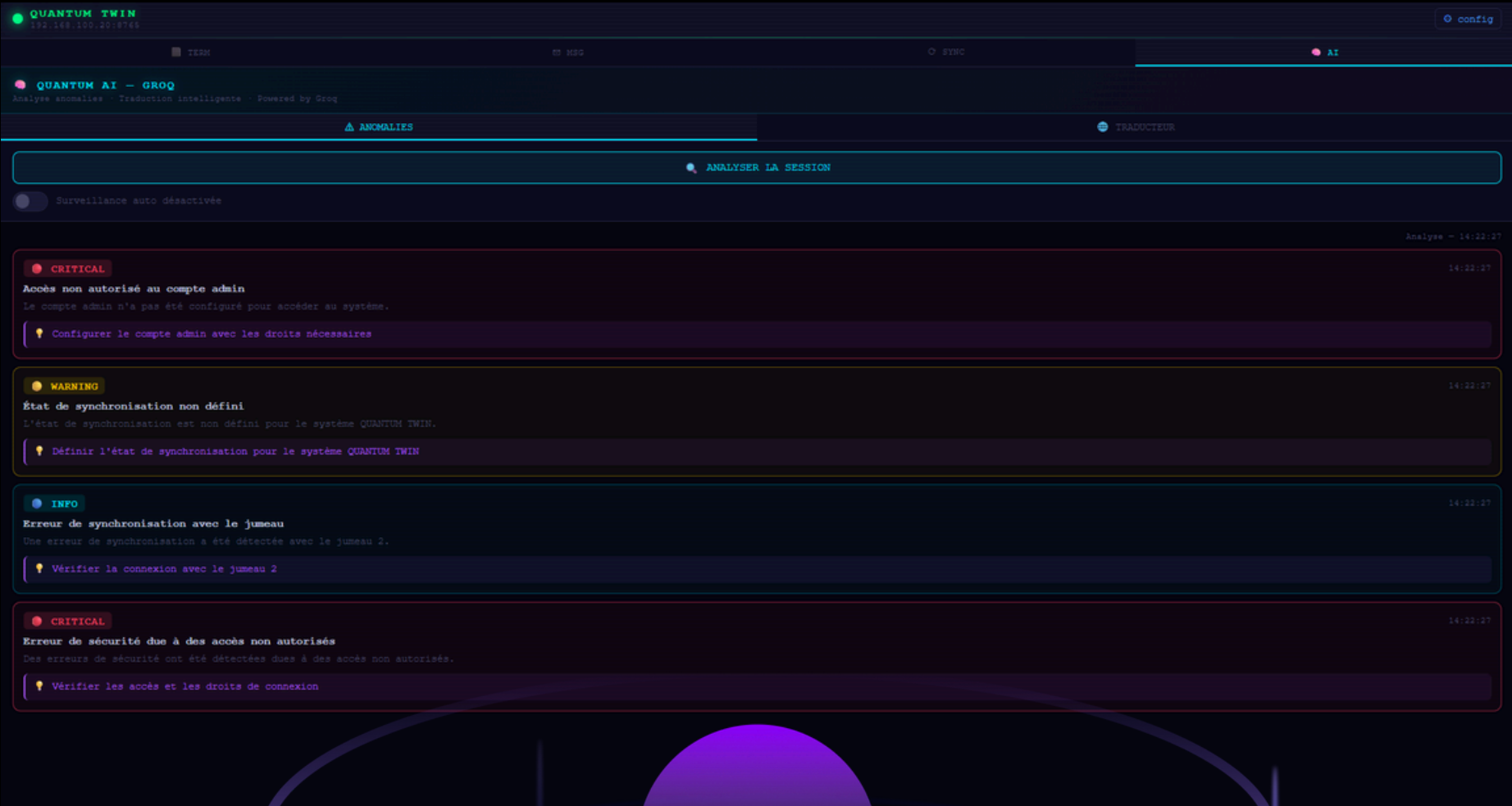
Architecture complète validée : sockets TCP/UDP, multithreading POSIX, synchronisation mutex, et bridge WebSocket. Prochaines étapes : sémaphores, chiffrement E2E, protocole enrichi.



Sémaphores



Commandes Avancées





TRADUCTION / ANALYSE DES ANOMALIES / HISTO

Architecture complète validée : sockets TCP/UDP, multithreading POSIX, synchronisation mutex, et bridge WebSocket. Prochaines étapes : sémaphores, chiffrement E2E, protocole enrichi.



Sémaphores



Commandes Avancées

